



Making Tail Constraints Bind: Two Silent Failure Modes in CVaR-Constrained Portfolio RL

Nigel Li^{1,2}

Yutong Han²

¹University of New South Wales, Sydney

²Columbia University, New York City



Abstract

Train a portfolio allocator with reinforcement learning (RL) to maximise return and it has no reason to avoid big losses. The usual fix is a Lagrangian penalty on Conditional Value-at-Risk (CVaR), the average loss in the worst weeks. We found two ways this fix can quietly do nothing. If the penalty and the reward are on different scales, the penalty weight stays at zero and the budget is never enforced, yet the tail numbers still improve, so the problem is easy to miss. And the choice of test can decide the winner, because a single stress window and a rolling walk-forward rank the same methods in opposite order. Our answer is a filter that checks each portfolio before it trades and moves it toward minimum variance when its forecast CVaR is over budget. Out of sample, the share of weeks over budget falls from **75.1% to 7.4%**. Across 40 walk-forward folds on two universes, the constraint lowers tail risk in **26 of the 27** folds where it binds ($p = 8.8 \times 10^{-6}$).

Why a Tail Limit

In the week of 16 March 2020 the S&P 500 fell about 15% and the VIX closed at a record 82.69. Our allocator trained only for return was still holding equities. The same allocator with a CVaR limit had already moved into Treasuries and cash, and its drawdown was **7.7% against 16.3%**.

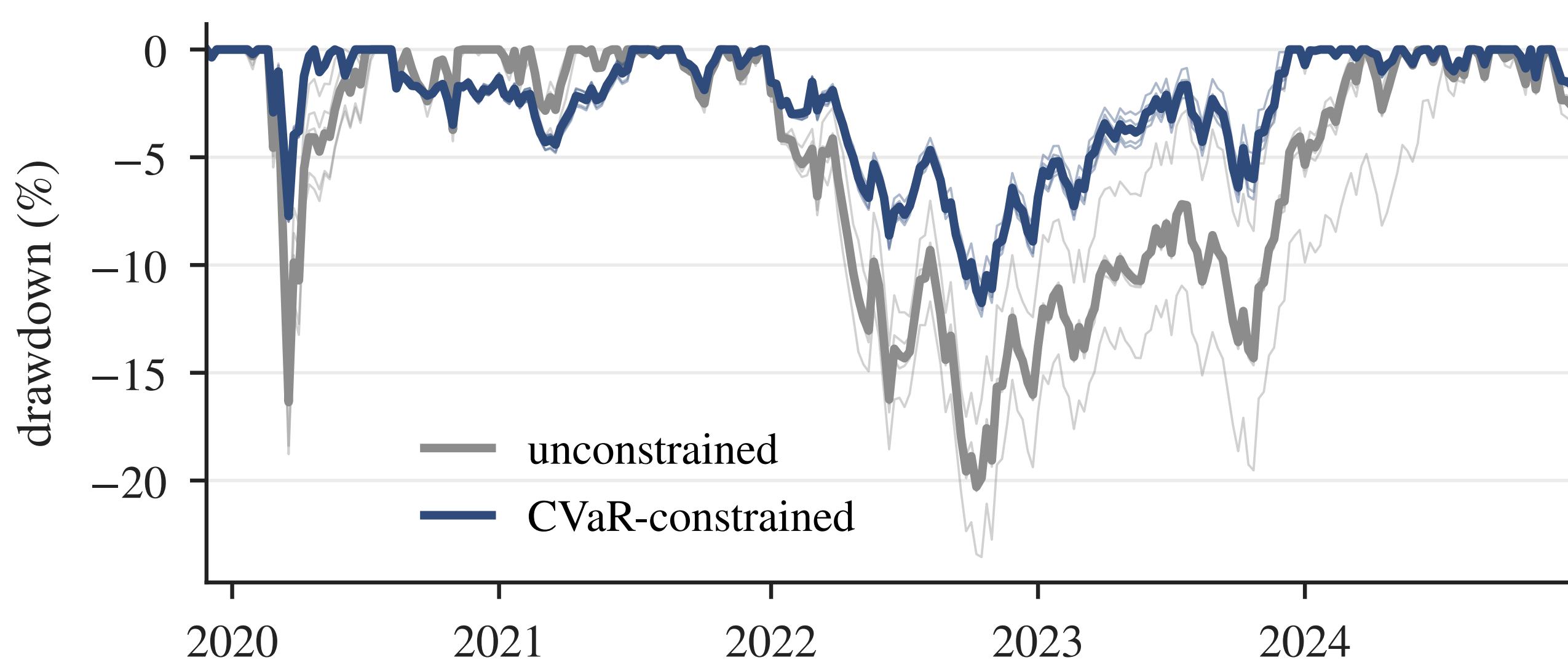
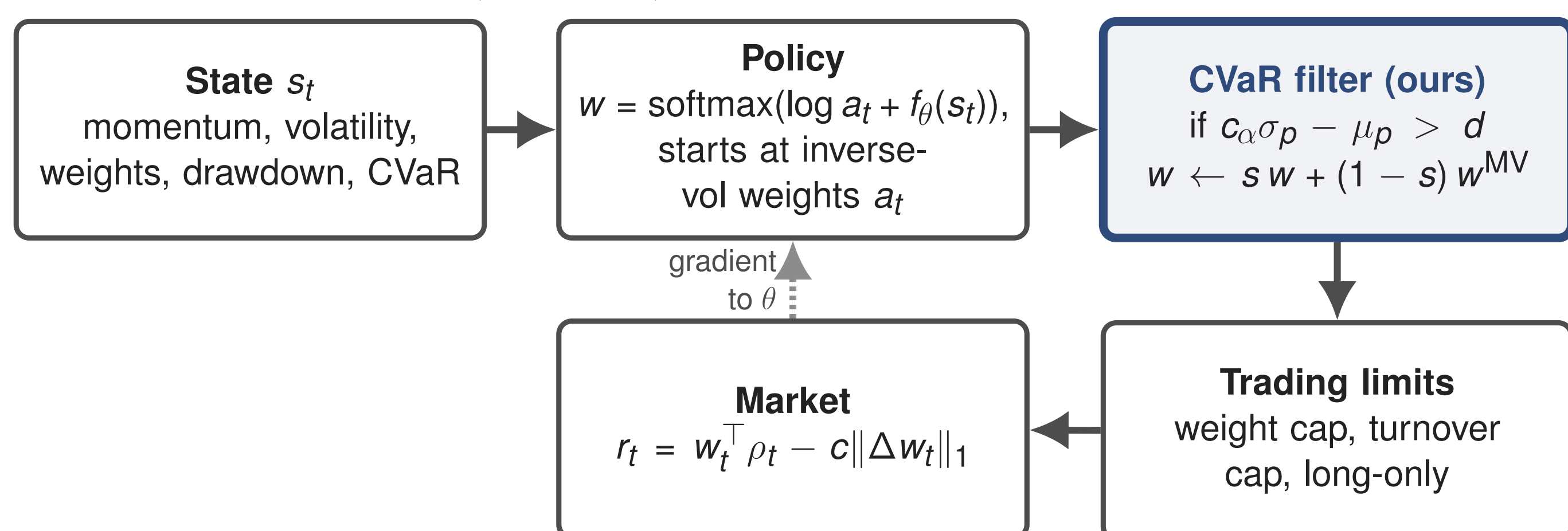


Figure 1. Drawdown over 2020–2024. Bold lines are the five-seed average.

How the Allocator Works

Every week the allocator splits money across seven ETFs that cover the main macro blocks (SPY, TLT, HYG, DBC, GLD, UUP, BIL). It is long-only, holds at most 40% in any one asset, has its weekly turnover capped at 0.50 and pays 5bps per trade. We use 887 weekly returns from 2008 to 2024. The budget caps $\text{CVaR}_{0.95}$, the average loss in the worst 5% of weeks, at $d = 1.2\%$. Training back-propagates straight through the portfolio return.

$$\mathcal{L} = -\text{Sharpe} + \lambda \text{ReLU}(\widehat{\text{CVaR}} - d) + \kappa \text{turnover}, \quad \lambda \leftarrow \max\{0, \lambda + \eta_{\lambda}(\widehat{\text{CVaR}} - d)\}$$



The filter estimates CVaR from the last 104 weeks of returns (Gaussian, $c_{0.95} = 2.06$) and keeps as much of the original portfolio as the budget allows. It only sees data up to the previous week, and a test confirms it cannot see the future.

Failure 1. The Penalty Never Switches On

The penalty weight λ is supposed to rise whenever the portfolio is over budget. In our setup the CVaR overshoot is around 10^{-2} , and with a step size of 10^{-3} each update nudges λ by about 10^{-5} . After 1500 updates it has barely moved. A model-free actor–critic did the same thing when its reward signal was standardised and its cost signal was left at 10^{-3} .

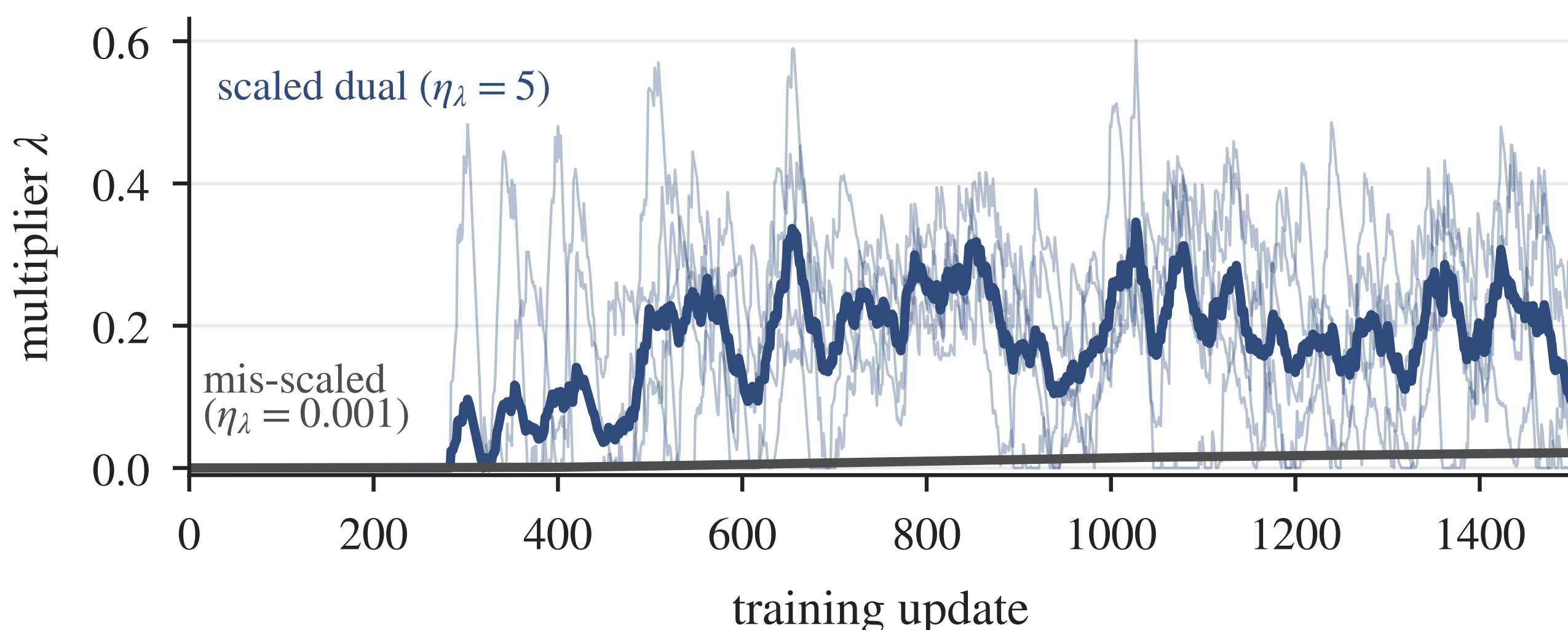


Figure 2. λ during training. Bold lines are the five-seed average.

Why is this easy to miss? The rest of the pipeline cuts CVaR_{99} by **61%** on its own, even with λ near zero ($p = 0.0044$). The tail numbers look good whether or not the penalty is doing anything.

☒ **Check this.** Plot λ during training. If it sits at zero, the constraint is switched off.

Failure 2. The Test Picks the Winner

On a 31-asset universe, a single stress test makes our learner look better than every classical optimiser (Sharpe 0.85 against 0.23 for minimum variance). Run the same comparison as a 20-fold rolling walk-forward and minimum variance comes out well ahead (1.40 against 0.74). Better co-variance estimates (Ledoit–Wolf) do not explain the gap. The tail-risk benefit over unconstrained RL shows up under both tests ($p = 6 \times 10^{-4}$). The win over the optimiser shows up in only one.

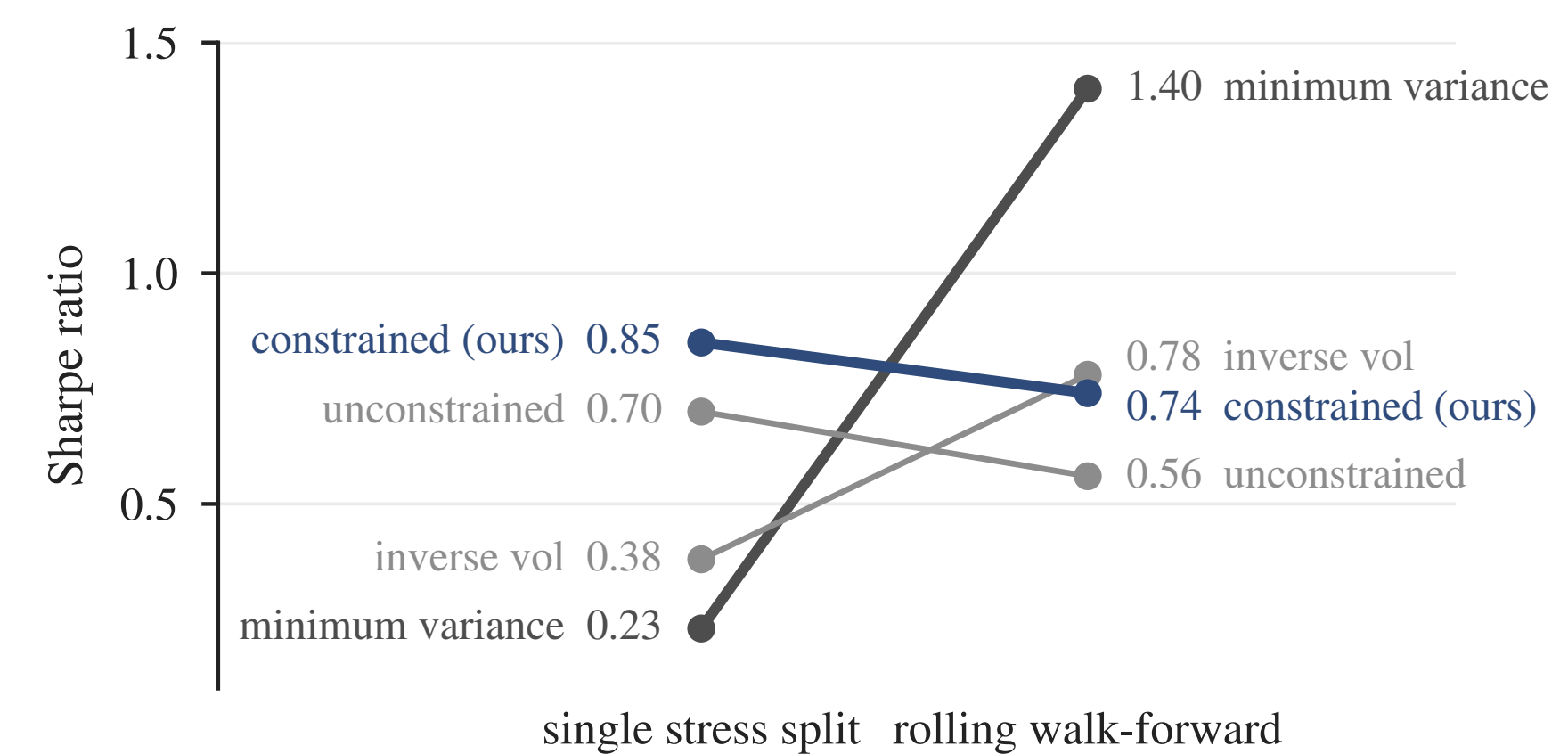


Figure 3. The same four methods ranked by two different tests.

☒ **Check this.** Compare learners with optimisers on rolling walk-forward folds and report Deflated Sharpe, not one test window.

The Fix. Check Every Trade

A larger step size makes the penalty work during training, but not on new data. The portfolio was still over budget in 75% of test weeks, and four different ways of setting the budget all landed between 71% and 78%. So instead of relying on the penalty alone, we added a check right before each trade.

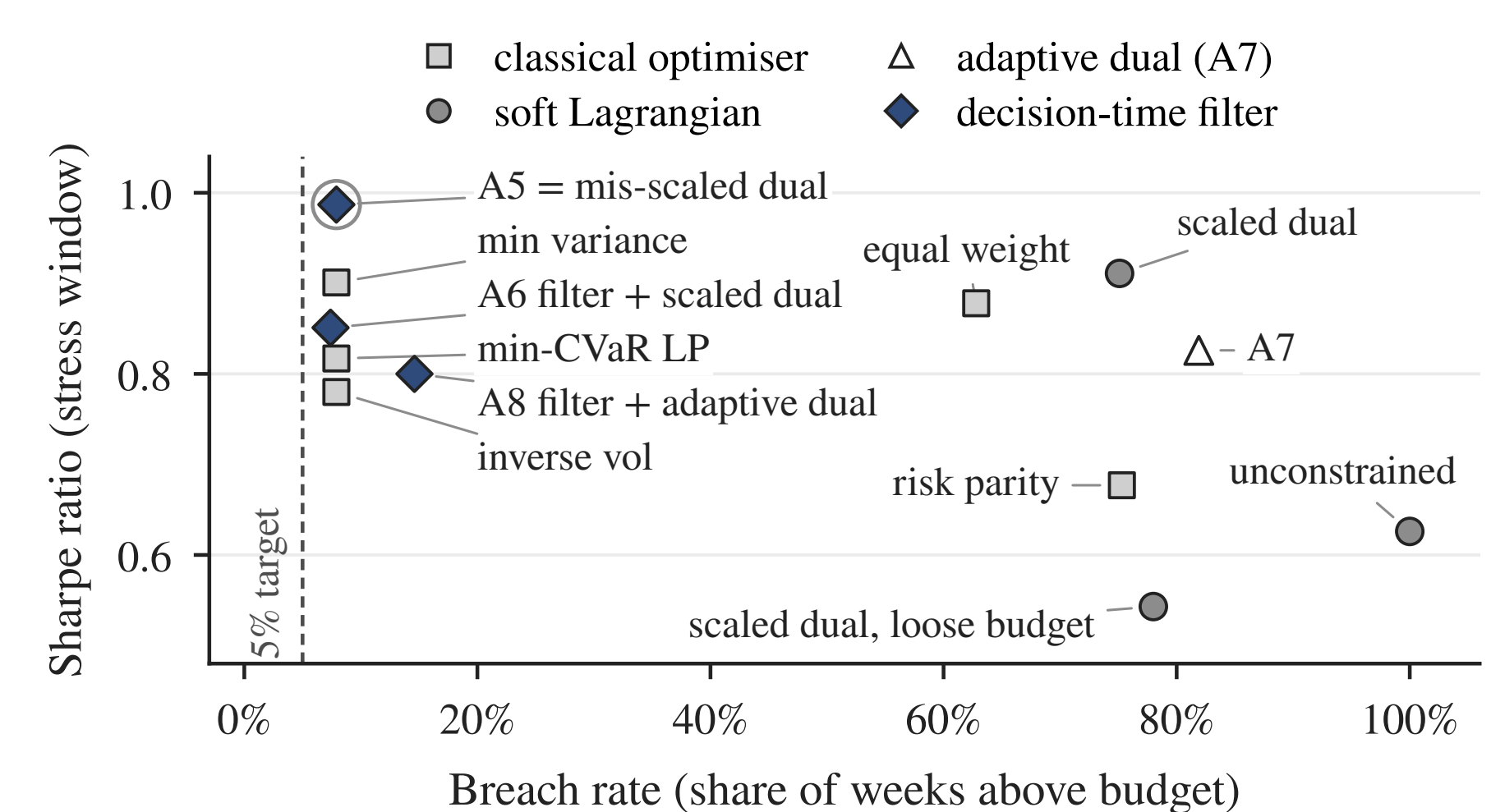


Figure 4. Share of weeks over budget against Sharpe. The dashed line is the 5% we allow.

variant (5-seed mean)	Sharpe	CVaR ₉₉	over budget
A1 no constraint	0.626	0.065	100.0%
A2 penalty, step too small	0.987	0.025	7.9%
A3 penalty, step fixed	0.911	0.028	75.1%
A7 penalty adjusted live	0.826	0.031	81.9%
A6 filter + penalty	0.851	0.016	7.4%
minimum variance	0.901		7.9%

A7 keeps raising λ during testing and is still over budget, so A6's gain comes from the filter.

Does It Hold Up?

We re-ran the comparison as a rolling walk-forward with 26-week test windows on two universes, measuring CVaR_{99} with and without the constraint.

universe	folds	lower tail risk	Wilcoxon p
macro ETFs (7)	20	9 (11 tied)	0.0077
sector ETFs (10)	20	17	0.0003
pooled	40	26 of 27 untied	8.8×10^{-6}
grouped by calendar	20		3.3×10^{-4}

A tie is a fold where the budget never binds, so both versions hold the same portfolio. Both universes pass a Benjamini–Hochberg correction. Re-running the stress test on sector ETFs gives the same picture, with CVaR_{99} down 30% and Sharpe unchanged.

Limitations and Takeaways

Our learner does not beat plain rolling minimum variance on Sharpe out of sample. Once you correct for the number of configurations tried (Deflated Sharpe), its Sharpe edge over classical rules disappears. What does hold up is lower tail risk and staying within budget. We only tested weekly data with simple trading costs, and the Gaussian forecast in the filter can underestimate fat tails. If you build one of these, four checks are worth doing.

- ☒ **Plot λ .** A flat line at zero means the constraint is off.
- ☒ **Put reward and cost on the same scale.** Standardise the cost signal and size the step to the CVaR overshoot.
- ☒ **Test on rolling folds** and report Deflated Sharpe before claiming you beat an optimiser.
- ☒ **Enforce the budget when you trade.** A penalty only discourages breaches on average.

